

UNIVERSIDAD POLITÉCNICA
DE BAJA CALIFORNIA

UNIDAD 1
EXTRACCIÓN DE DATOS DQL

Reporte técnico de ejercicios prácticos
SQL Server · Consultas multitabla JOIN

DATOS ACADÉMICOS	INFORMACIÓN
Asignatura	BASES DE DATOS AVANZADAS
Docente	Alicia del Refugio López Aguirre
Estudiante	Peyman Miyandashti
Programa	Ingeniería en Tecnologías de la Información e Innovación Digital
Grupo / periodo	ITIID 5AFM 2026-3
Fecha de elaboración	6 de noviembre de 2026
Actividad	Unidad 1: Extracción de Datos DQL

Contenido: introducción, objetivos, fundamentos DQL, tres ejercicios resueltos con preguntas, código, resultados esperados y análisis, conclusiones, referencias y anexos de código fuente.

Documento elaborado a partir de los archivos SQL entregados y del enunciado de la actividad. Los resultados tabulados se deducen de los registros INSERT; no se afirma una ejecución en una instancia de SQL Server.



1. Introducción

El lenguaje SQL permite almacenar, consultar y relacionar información estructurada. Esta práctica se centra en DQL (Data Query Language), especialmente en la instrucción SELECT y en operaciones JOIN para recuperar datos distribuidos en varias tablas. Los tres casos plantean escenarios cotidianos: clientes y provincias, inscripciones e inasistencias deportivas, y estudiantes con cursos.

Además de presentar cada enunciado, se documenta la preparación de las tablas, la carga de datos, las consultas utilizadas, sus respuestas esperadas y la interpretación de los resultados. Los scripts originales incluyen también instrucciones DDL (CREATE/DROP) y DML (INSERT), necesarias para preparar la información sobre la que opera DQL.

1.1 Objetivo general

Aplicar consultas multitabla mediante INNER JOIN y LEFT JOIN en SQL Server para integrar información de tablas relacionadas, seleccionar columnas específicas, filtrar registros y ordenar resultados.

1.2 Objetivos específicos

• Crear estructuras de datos con claves primarias, identificadores automáticos y, cuando corresponda, claves foráneas.

• Insertar datos de ejemplo conservando la correspondencia entre entidades.

• Utilizar alias y cláusulas ON, WHERE y ORDER BY para obtener respuestas verificables.

• Comprender las uniones con condiciones simples y compuestas y la diferencia entre INNER JOIN y LEFT JOIN.

• Interpretar resultados, detectar discrepancias del enunciado y comunicar conclusiones técnicas.



1.3 Conceptos indispensables

Elemento	Función
SELECT	Selecciona columnas y recupera datos.
FROM	Indica tabla de origen.
INNER JOIN	Devuelve coincidencias en ambas tablas.
LEFT JOIN	Conserva todas las filas de la tabla izquierda, aun sin coincidencia.
ON	Define la condición de relación.
WHERE	Filtra registros bajo una condición.
ORDER BY	Ordena el resultado según una o varias columnas.
IDENTITY / PRIMARY KEY	Autonumeración e identificación de registros.
FOREIGN KEY	Declara una referencia entre tablas.

2. Ejercicio 1 · Clientes y provincias

2.1 Contexto y pregunta de la actividad

Pregunta. ¿Cómo consultar clientes junto con el nombre de su provincia, ordenar por provincia y recuperar únicamente los de Santa Fe?

Respuesta. Relacionando clientes.codigoprovincia con provincias.codigo mediante INNER JOIN; después se utiliza ORDER BY o WHERE según el resultado solicitado.

2.2 Estructura y preparación de datos

Se emplean dos tablas: provincias (catálogo con código y nombre) y clientes (datos personales básicos y código de provincia). En el archivo entregado se crea la base EJERCICIO1_CLIENTES, se eliminan las tablas existentes y se crean de nuevo. La tabla provincias se crea antes de clientes. Los campos codigo utilizan IDENTITY y son claves primarias. La relación es lógica por codigoprovincia, pero el script no declara una FOREIGN KEY en clientes.




```
CREATE DATABASE EJERCICIO1_CLIENTES;
USE EJERCICIO1_CLIENTES;
IF OBJECT_ID('clientes', 'U') IS NOT NULL DROP TABLE clientes;
IF OBJECT_ID('provincias', 'U') IS NOT NULL DROP TABLE provincias;
CREATE TABLE provincias (
    codigo TINYINT IDENTITY,
    nombre VARCHAR(20),
    PRIMARY KEY (codigo)
);
CREATE TABLE clientes (
    codigo INT IDENTITY,
    nombre VARCHAR(30), domicilio VARCHAR(30),
    ciudad VARCHAR(20), codigoprovincia TINYINT NOT NULL,
    PRIMARY KEY (codigo)
);
```

Se insertan tres provincias y siete clientes. Como los códigos son IDENTITY, las provincias quedan numeradas 1 = Cordoba, 2 = Santa Fe y 3 = Corrientes, suponiendo tablas recién creadas y sin cambios adicionales.

Código	Provincia	Número de clientes
1	Cordoba	4
2	Santa Fe	2
3	Corrientes	1

2.3 Pregunta 4 · Datos de ambas tablas usando alias

Pregunta. ¿Qué consulta muestra el cliente, domicilio, ciudad y provincia utilizando alias?

Respuesta. El alias c identifica clientes y p identifica provincias; la igualdad de códigos establece la unión.

```
SELECT c.nombre AS Cliente, c.domicilio AS Domicilio,
       c.ciudad AS Ciudad, p.nombre AS Provincia
FROM clientes AS c
INNER JOIN provincias AS p ON c.codigoprovincia = p.codigo;
```

Cliente	Domicilio	Ciudad	Provincia
Lopez Marcos	Colon 111	Córdoba	Cordoba
Perez Ana	San Martin 222	Cruz del Eje	Cordoba
Garcia Juan	Rivadavia 333	Villa Maria	Cordoba
Perez Luis	Sarmiento 444	Rosario	Santa Fe
Pereyra Lucas	San Martin 555	Cruz del Eje	Cordoba



Cliente	Domicilio	Ciudad	Provincia
Gomez Ines	San Martin 666	Santa Fe	Santa Fe
Torres Fabiola	Alem 777	Ibera	Corrientes

Resultado esperado: 7 filas y 4 columnas. Sin ORDER BY, SQL Server no garantiza un orden específico de presentación.

2.4 Pregunta 5 · Orden por nombre de provincia

Pregunta. ¿Cómo mostrar la misma información ordenada por nombre de provincia?

Respuesta. Se agrega ORDER BY p.nombre. Las siete filas se agrupan visualmente por los nombres de las provincias en orden ascendente.

```
SELECT c.nombre AS Cliente, c.domicilio AS Domicilio,
       c.ciudad AS Ciudad, p.nombre AS Provincia
FROM clientes AS c
INNER JOIN provincias AS p ON c.codigoprovincia = p.codigo
ORDER BY p.nombre;
```

Resultado esperado: 7 filas: primero los 4 clientes de Cordoba, después 1 de Corrientes y finalmente 2 de Santa Fe, bajo una ordenación alfabética usual. El orden entre clientes de la misma provincia no está definido porque no se añadió un segundo criterio.

2.5 Pregunta 6 · Clientes de Santa Fe

Pregunta. ¿Qué clientes pertenecen a la provincia Santa Fe?

Respuesta. Se utiliza WHERE p.nombre = 'Santa Fe'. El resultado contiene Perez Luis y Gomez Ines.

```
SELECT c.nombre AS Cliente, c.domicilio AS Domicilio,
       c.ciudad AS Ciudad, p.nombre AS Provincia
FROM clientes AS c
INNER JOIN provincias AS p ON c.codigoprovincia = p.codigo
WHERE p.nombre = 'Santa Fe';
```

Cliente	Domicilio	Ciudad	Provincia
Perez Luis	Sarmiento 444	Rosario	Santa Fe
Gomez Ines	San Martin 666	Santa Fe	Santa Fe

Interpretación: la consulta devuelve exactamente 2 registros, lo solicitado. El filtro utiliza la provincia del catálogo y no exige que la columna ciudad sea también Santa Fe.



2.6 Conclusión del ejercicio 1

Se comprobó, a partir de los datos suministrados, que INNER JOIN combina el nombre de una provincia con cada cliente que referencia su código. Los alias simplifican la escritura; ORDER BY organiza la presentación; WHERE restringe los resultados sin modificar los registros originales. Como mejora de integridad, sería conveniente definir FOREIGN KEY sobre clientes.codigoprovincia.

3. Ejercicio 2 · Inscriptos e inasistencias

3.1 Contexto y pregunta de la actividad

Pregunta. ¿Cómo asociar las fechas de ausencia con la persona correcta y el deporte correcto?

Respuesta. La unión requiere dos condiciones simultáneas: igualdad de documento e igualdad de deporte. Un mismo participante puede estar inscrito en varias disciplinas.

3.2 Tablas, clave compuesta y registros

La tabla inscriptos almacena nombre, documento, deporte y matrícula; la clave primaria es compuesta (documento, deporte). En matrícula, s significa pagada y n significa impaga. La tabla inasistencias almacena documento, deporte y fecha. Se registran seis inscripciones y seis ausencias. El archivo SQL elimina primero inasistencias y luego inscriptos antes de recrearlas.

```
IF OBJECT_ID('inasistencias') IS NOT NULL DROP TABLE inasistencias;
IF OBJECT_ID('inscriptos') IS NOT NULL DROP TABLE inscriptos;
CREATE TABLE inscriptos (
    nombre VARCHAR(30), documento CHAR(8),
    deporte VARCHAR(15), matricula CHAR(1),
    PRIMARY KEY (documento, deporte)
);
CREATE TABLE inasistencias (
    documento CHAR(8), deporte VARCHAR(15), fecha DATETIME
);
```

Inscripto	Deporte(s)	Matrícula
Juan Perez	tenis / natacion	s / s
Maria Lopez	tenis / natacion	s / n
Agustin Juarez	tenis	n
Marta Garcia	natacion	s

3.3 Pregunta 3 · Todas las inasistencias

Pregunta. ¿Cómo presentar nombre, deporte y fechas de inasistencia, ordenados por nombre y deporte?



Respuesta. Se usa INNER JOIN con ON compuesto por documento y deporte, y ORDER BY sobre nombre y deporte.

```
SELECT i.nombre, i.deporte, a.fecha
FROM inscriptos AS i
INNER JOIN inasistencias AS a
    ON i.documento = a.documento
   AND i.deporte = a.deporte
ORDER BY i.nombre, i.deporte;
```

Nombre	Deporte	Fecha
Agustin Juarez	tenis	2006-12-08
Juan Perez	natacion	2006-12-02
Juan Perez	tenis	2006-12-01
Juan Perez	tenis	2006-12-08
Maria Lopez	natacion	2006-12-02
Maria Lopez	tenis	2006-12-01

Resultado esperado: 6 registros de inasistencia relacionados con sus inscripciones. Marta Garcia no aparece porque no tiene ausencias registradas y INNER JOIN solo muestra coincidencias.

3.4 Pregunta 4 · Un inscripto en un deporte

Pregunta. ¿Qué ausencias tiene Juan Perez en tenis?

Respuesta. Se filtra por nombre Juan Perez y deporte tenis, manteniendo las dos condiciones de unión.

```
SELECT i.nombre, i.deporte, a.fecha
FROM inscriptos AS i
INNER JOIN inasistencias AS a
    ON i.documento = a.documento
   AND i.deporte = a.deporte
WHERE i.nombre = 'Juan Perez' AND i.deporte = 'tenis';
```

Nombre	Deporte	Fecha
Juan Perez	tenis	2006-12-01
Juan Perez	tenis	2006-12-08

Observación crítica: el enunciado anota «3 registros», pero los datos y la consulta del archivo proporcionado devuelven 2 para Juan Perez en tenis. Juan Perez tiene una tercera ausencia en natacion (2006-12-02); no pertenece al deporte filtrado. Para obtener tres ausencias de Juan Perez habría que quitar el filtro del deporte, lo cual ya no respondería la pregunta original.



3.5 Pregunta 5 · Matrícula pagada

Pregunta. ¿Qué ausencias corresponden a inscripciones cuya matrícula está pagada?

Respuesta. WHERE i.matricula = 's' filtra las inscripciones pagadas después de realizar la unión por documento y deporte.

```
SELECT i.nombre, i.deporte, a.fecha
FROM inscriptos AS i
INNER JOIN inasistencias AS a
    ON i.documento = a.documento
   AND i.deporte = a.deporte
WHERE i.matricula = 's'
ORDER BY i.nombre, i.deporte;
```

Nombre	Deporte	Fecha
Juan Perez	natacion	2006-12-02
Juan Perez	tenis	2006-12-01
Juan Perez	tenis	2006-12-08
Maria Lopez	tenis	2006-12-01

Resultado esperado: 4 registros, como pide el enunciado. Marta Garcia sí pagó, pero no registra inasistencias; Maria Lopez aparece solo en tenis, porque su matrícula de natación es impaga.

3.6 Conclusión del ejercicio 2

La clave compuesta evidencia por qué una persona puede tener más de una inscripción. Usar solo documento en JOIN daría asociaciones incorrectas entre deportes; agregar deporte preserva la correspondencia exacta. La cláusula WHERE permite analizar la situación de pago para cada inscripción, no solo para cada persona. También se identificó una discrepancia cuantitativa entre la consigna y los registros facilitados.

4. Ejercicio 3 · Estudiantes y cursos

4.1 Propuesta de base de datos

Pregunta. ¿Qué ejemplo de dos tablas permite desarrollar al menos tres consultas multitabla con JOIN?

Respuesta. Se propone un sistema académico simplificado con cursos y estudiantes. Cada estudiante tiene un codigocurso que referencia a cursos.codigo. Se registran cuatro cursos y cinco estudiantes.



4.2 Definición de tablas

```
IF OBJECT_ID('estudiantes') IS NOT NULL DROP TABLE estudiantes;
IF OBJECT_ID('cursos') IS NOT NULL DROP TABLE cursos;
CREATE TABLE cursos (
    codigo INT PRIMARY KEY,
    nombre VARCHAR(50)
);
CREATE TABLE estudiantes (
    codigo INT PRIMARY KEY,
    nombre VARCHAR(50), edad INT, codigocurso INT,
    FOREIGN KEY (codigocurso) REFERENCES cursos(codigo)
);
```

A diferencia del ejercicio 1, aquí la relación se protege explícitamente con FOREIGN KEY, de forma que un codigocurso distinto de los existentes sea rechazado por el motor al intentar insertarse.

Código	Curso	Estudiantes asignados
1	Matematicas	Juan Perez, Pedro Ramirez
2	Programacion	Maria Lopez, Ana Torres
3	Base de Datos	Carlos Garcia
4	Ingles	Ninguno

4.3 Consulta 1 · Todos los estudiantes con su curso

Pregunta. ¿Cuáles son los nombres y edades de los estudiantes junto con el curso que llevan?

Respuesta. Se utiliza INNER JOIN para enlazar el curso identificado por codigocurso.

```
SELECT e.nombre AS Estudiante, e.edad AS Edad,
       c.nombre AS Curso
FROM estudiantes AS e
INNER JOIN cursos AS c ON e.codigocurso = c.codigo;
```

Estudiante	Edad	Curso
Juan Perez	20	Matematicas
Maria Lopez	22	Programacion
Carlos Garcia	21	Base de Datos
Ana Torres	19	Programacion
Pedro Ramirez	23	Matematicas



Resultado esperado: 5 filas correspondientes a los cinco estudiantes. La consulta conserva únicamente las relaciones existentes, por lo que el curso Ingles no aparece en esta lista de estudiantes.

4.4 Consulta 2 · Estudiantes de Programacion

Pregunta. ¿Quiénes están inscritos en el curso Programacion?

Respuesta. Se incorpora WHERE c.nombre = 'Programacion' para filtrar el curso.

```
SELECT e.nombre AS Estudiante, c.nombre AS Curso
FROM estudiantes AS e
INNER JOIN cursos AS c ON e.codigocurso = c.codigo
WHERE c.nombre = 'Programacion';
```

Estudiante	Curso
Maria Lopez	Programacion
Ana Torres	Programacion

Resultado esperado: 2 estudiantes: Maria Lopez y Ana Torres.

4.5 Consulta 3 · Todos los cursos, incluso los vacíos

Pregunta. ¿Cómo listar cursos que todavía no tienen estudiantes?

Respuesta. Se utiliza LEFT JOIN desde cursos hacia estudiantes. El curso sin correspondencia permanece y la columna estudiante aparece como NULL.

```
SELECT c.nombre AS Curso, e.nombre AS Estudiante
FROM cursos AS c
LEFT JOIN estudiantes AS e ON c.codigo = e.codigocurso
ORDER BY c.nombre;
```

Curso	Estudiante
Base de Datos	Carlos Garcia
Ingles	NULL
Matematicas	Juan Perez
Matematicas	Pedro Ramirez
Programacion	Maria Lopez
Programacion	Ana Torres

Resultado esperado: 6 filas. La fila Ingles / NULL demuestra la diferencia clave: LEFT JOIN conserva cursos sin estudiantes, mientras que INNER JOIN los excluiría. Entre estudiantes de un mismo curso no se



garantiza el orden al usar solamente ORDER BY c.nombre.

4.6 Conclusión del ejercicio 3

El ejemplo académico permite comparar dos tipos de unión. INNER JOIN es apropiado cuando se requieren solo asociaciones existentes; LEFT JOIN es indispensable cuando también interesa identificar categorías sin registros asociados. La FOREIGN KEY ofrece protección adicional contra referencias inválidas y contribuye a mantener consistente la base de datos.

5. Análisis comparativo de los tres ejercicios

Caso	Unión y condiciones	Filas esperadas
1. Clientes/provincias	INNER JOIN sobre código; ordenar y filtrar por provincia	7 / 7 / 2
2. Deporte/ausencias	INNER JOIN con documento + deporte; filtrar persona y matrícula	6 / 2 / 4
3. Estudiantes/cursos	INNER JOIN, INNER JOIN + WHERE, LEFT JOIN	5 / 2 / 6

La precisión de una consulta depende de la relación definida en ON. Cuando la identificación requiere varios campos, omitir uno de ellos puede multiplicar filas o asociar datos que no corresponden. Además, el resultado de una consulta es una vista de lectura: SELECT, WHERE y ORDER BY no cambian por sí mismos las tablas almacenadas.

5.1 Buenas prácticas identificadas

• Especificar las columnas y utilizar alias descriptivos al combinar tablas.

• Definir FOREIGN KEY cuando una columna contiene referencias a otra tabla.

• No asumir que las filas tienen orden estable sin ORDER BY.

• Validar cada cantidad esperada contra los datos insertados en lugar de copiar cifras del enunciado.

• Al ejecutar scripts de recreación, considerar que DROP TABLE elimina datos existentes; utilizar entornos de práctica.

• En SQL Server, GO es un separador de lotes reconocido por herramientas como SSMS, no una sentencia T-SQL.



6. Conclusión general personal

Con esta actividad comprendí que una base de datos relacional no consiste únicamente en guardar información en tablas separadas, sino en diseñar vínculos correctos entre ellas para generar respuestas útiles. En el primer ejercicio logré interpretar cómo recuperar los datos del cliente junto con la provincia y cómo filtrar las filas de Santa Fe. En el segundo aprendí que la identificación de una inscripción requiere tanto el documento como el deporte y que una condición incompleta puede producir resultados erróneos. En el tercer ejercicio comprobé conceptualmente que INNER JOIN y LEFT JOIN resuelven necesidades distintas, porque este último muestra incluso los cursos sin estudiantes.

También aprendí a distinguir las instrucciones utilizadas para preparar el ambiente (CREATE, DROP e INSERT) de las instrucciones DQL que extraen información (SELECT). Analizar los registros de ejemplo me permitió revisar los resultados esperados y detectar que una de las cantidades indicadas en la consigna no coincide con los datos. Considero que estas habilidades son importantes para construir sistemas escolares, comerciales y administrativos más confiables, ya que facilitan la consulta organizada y consistente de la información.

7. Fuentes y alcance de verificación

• Enunciado de «Ejercicios para el estudiante», Unidad 1 Extracción de Datos DQL, proporcionado con esta actividad.

• Archivos entregados: Ejercicio1_Clientes_Provincias(1).sql, Ejercicio2_Inscriptos_Inasistencias(1).sql y Ejercicio3_Consultas_Multitablas(1).sql.

• Plataforma didáctica: Google Classroom, asignatura Bases de Datos Avanzadas, ITIID 5AFM 2026-3.

Nota metodológica: Los cuadros de resultados son salidas esperadas razonadas a partir de los INSERT, no capturas ni certificaciones de ejecución real. El archivo del ejercicio 1 contiene CREATE DATABASE y USE, y el separador GO; en el ejercicio 2 y 3 se usa el contexto de base de datos ya seleccionado. Para una ejecución real se debe revisar que la base de datos activa sea la correcta.

Anexo A · Código SQL original del ejercicio 1

Transcripción completa del archivo entregado, conservando las instrucciones y consultas. Las tablas de resultados se muestran en el desarrollo anterior.




```
CREATE DATABASE EJERCICIO1_CLIENTES;
USE EJERCICIO1_CLIENTES;

IF OBJECT_ID('clientes', 'U') IS NOT NULL
    DROP TABLE clientes;

IF OBJECT_ID('provincias', 'U') IS NOT NULL
    DROP TABLE provincias;

CREATE TABLE provincias (
    codigo TINYINT IDENTITY,
    nombre VARCHAR(20),
    PRIMARY KEY (codigo)
);

CREATE TABLE clientes (
    codigo INT IDENTITY,
    nombre VARCHAR(30),
    domicilio VARCHAR(30),
    ciudad VARCHAR(20),
    codigoprovincia TINYINT NOT NULL,
    PRIMARY KEY (codigo)
);
GO

INSERT INTO provincias (nombre)
VALUES ('Cordoba');

INSERT INTO provincias (nombre)
VALUES ('Santa Fe');

INSERT INTO provincias (nombre)
VALUES ('Corrientes');

INSERT INTO clientes
VALUES ('Lopez Marcos','Colon 111','Córdoba',1);

INSERT INTO clientes
VALUES ('Perez Ana','San Martin 222','Cruz del Eje',1);

INSERT INTO clientes
VALUES ('Garcia Juan','Rivadavia 333','Villa Maria',1);
```




```
INSERT INTO clientes
VALUES ('Perez Luis','Sarmiento 444','Rosario',2);

INSERT INTO clientes
VALUES ('Pereyra Lucas','San Martin 555','Cruz del Eje',1);

INSERT INTO clientes
VALUES ('Gomez Ines','San Martin 666','Santa Fe',2);

INSERT INTO clientes
VALUES ('Torres Fabiola','Alem 777','Ibera',3);


SELECT
    c.nombre AS Cliente,
    c.domicilio AS Domicilio,
    c.ciudad AS Ciudad,
    p.nombre AS Provincia
FROM clientes AS c
INNER JOIN provincias AS p
    ON c.codigoprovincia = p.codigo;


SELECT
    c.nombre AS Cliente,
    c.domicilio AS Domicilio,
    c.ciudad AS Ciudad,
    p.nombre AS Provincia
FROM clientes AS c
INNER JOIN provincias AS p
    ON c.codigoprovincia = p.codigo
ORDER BY p.nombre;


SELECT
    c.nombre AS Cliente,
    c.domicilio AS Domicilio,
    c.ciudad AS Ciudad,
    p.nombre AS Provincia
FROM clientes AS c
INNER JOIN provincias AS p
```

```
    ON c.codigoprovincia = p.codigo
WHERE p.nombre = 'Santa Fe';
```



Anexo B · Código SQL original del ejercicio 2

```
IF OBJECT_ID('inasistencias') IS NOT NULL
    DROP TABLE inasistencias;

IF OBJECT_ID('inscriptos') IS NOT NULL
    DROP TABLE inscriptos;

CREATE TABLE inscriptos (
    nombre VARCHAR(30),
    documento CHAR(8),
    deporte VARCHAR(15),
    matricula CHAR(1),
    PRIMARY KEY (documento, deporte)
);

CREATE TABLE inasistencias (
    documento CHAR(8),
    deporte VARCHAR(15),
    fecha DATETIME
);

INSERT INTO inscriptos VALUES ('Juan Perez','22222222','tenis','s');
INSERT INTO inscriptos VALUES ('Maria Lopez','23333333','tenis','s');
INSERT INTO inscriptos VALUES ('Agustin Juarez','24444444','tenis','n');
INSERT INTO inscriptos VALUES ('Marta Garcia','25555555','natacion','s');
INSERT INTO inscriptos VALUES ('Juan Perez','22222222','natacion','s');
INSERT INTO inscriptos VALUES ('Maria Lopez','23333333','natacion','n');

INSERT INTO inasistencias VALUES ('22222222','tenis','2006-12-01');
INSERT INTO inasistencias VALUES ('22222222','tenis','2006-12-08');
INSERT INTO inasistencias VALUES ('23333333','tenis','2006-12-01');
INSERT INTO inasistencias VALUES ('24444444','tenis','2006-12-08');
INSERT INTO inasistencias VALUES ('22222222','natacion','2006-12-02');
INSERT INTO inasistencias VALUES ('23333333','natacion','2006-12-02');

SELECT
    i.nombre,
    i.deporte,
    a.fecha
FROM inscriptos AS i
INNER JOIN inasistencias AS a
    ON i.documento = a.documento
    AND i.deporte = a.deporte
```




```
ORDER BY i.nombre, i.deporte;

SELECT
    i.nombre,
    i.deporte,
    a.fecha
FROM inscriptos AS i
INNER JOIN inasistencias AS a
    ON i.documento = a.documento
    AND i.deporte = a.deporte
WHERE i.nombre = 'Juan Perez'
    AND i.deporte = 'tenis';

SELECT
    i.nombre,
    i.deporte,
    a.fecha
FROM inscriptos AS i
INNER JOIN inasistencias AS a
    ON i.documento = a.documento
    AND i.deporte = a.deporte
WHERE i.matricula = 's'
ORDER BY i.nombre, i.deporte;
```



Anexo C · Código SQL original del ejercicio 3

```
IF OBJECT_ID('estudiantes') IS NOT NULL
    DROP TABLE estudiantes;

IF OBJECT_ID('cursos') IS NOT NULL
    DROP TABLE cursos;

CREATE TABLE cursos (
    codigo INT PRIMARY KEY,
    nombre VARCHAR(50)
);

CREATE TABLE estudiantes (
    codigo INT PRIMARY KEY,
    nombre VARCHAR(50),
    edad INT,
    codigocurso INT,
    FOREIGN KEY (codigocurso) REFERENCES cursos(codigo)
);

INSERT INTO cursos VALUES (1, 'Matematicas');
INSERT INTO cursos VALUES (2, 'Programacion');
INSERT INTO cursos VALUES (3, 'Base de Datos');
INSERT INTO cursos VALUES (4, 'Ingles');

INSERT INTO estudiantes VALUES (1, 'Juan Perez', 20, 1);
INSERT INTO estudiantes VALUES (2, 'Maria Lopez', 22, 2);
INSERT INTO estudiantes VALUES (3, 'Carlos Garcia', 21, 3);
INSERT INTO estudiantes VALUES (4, 'Ana Torres', 19, 2);
INSERT INTO estudiantes VALUES (5, 'Pedro Ramirez', 23, 1);

SELECT
    e.nombre AS Estudiante,
    e.edad AS Edad,
    c.nombre AS Curso
FROM estudiantes AS e
INNER JOIN cursos AS c
    ON e.codigocurso = c.codigo;

SELECT
    e.nombre AS Estudiante,
    c.nombre AS Curso
FROM estudiantes AS e
```




```
INNER JOIN cursos AS c
    ON e.codigocurso = c.codigo
WHERE c.nombre = 'Programacion';

SELECT
    c.nombre AS Curso,
    e.nombre AS Estudiante
FROM cursos AS c
LEFT JOIN estudiantes AS e
    ON c.codigo = e.codigocurso
ORDER BY c.nombre;
```